专升本 Java 程序设计模拟卷-卷 3

深度解析讲义

作者：fifine-专升本

日期：2026 年 3 月 2 日

题号	一 (40 分)	二 (20 分)	三 (20 分)	四 (30 分)	五 (40 分)	总分 (150 分)
得分						

一、单项选择题（本大题共 20 小题，每小题 2 分，共 40 分）

1. 下列哪个是 **Java** 中合法的标识符？

   A. 2variable

   B. _myVar

   C. my-var

   D. class

答案与深度解析 答案：**B. _myVar**

【核心认知框架】

- 题目本质：考查 Java 标识符命名规范（JLS §3.8）
- 关键区分点：首字符限制、关键字冲突、特殊符号允许性
- 命题人意图：测试基础语法敏感度，区分标识符与关键字

【选项原理深度拆解】

2. **A. 2variable** —错误（数字开头）
   - **JLS** 规范：JLS §3.8 规定标识符首字符必须是 JavaLetter
   - 字节码证据：

```java
int 2variable = 1; // □□□□□illegal start of type
// javap □□□□□□□□□□
```

   - 正则规则：标识符匹配 `[JavaLetter][JavaLetterOrDigit]*`

3. **B. _myVar** —正确（合法标识符）

fifine专升本

- **JLS** 规范：下划线 _ 和美元符号 $ 是合法的起始字符
- 字节码证据：

```
int _myVar = 1;
// javap -c:
// 0: iconst_1
// 1: istore_1 // □□□□□□□□□□
```

- 命名惯例：虽合法，但现代 Java 规范不建议以 _ 开头（易混淆）

4. **C. my-var** —错误（含减号）
   - 词法分析：减号 - 是运算符，会被解析为 my - var
   - 编译错误：';' expected 或 not a statement
   - 合法替代：使用驼式命名 myVar
5. **D. class** —错误（关键字冲突）
   - **JLS** 规范：class 是保留关键字（JLS §3.9）
   - 编译错误：class, interface, or enum expected
   - 例外：Java 9+ 中 var 是类型推断关键字，也不能作标识符

【应背诵知识点】
- 首字符：字母、_、$
- 后续字符：字母、数字、_、$
- 禁止：关键字、数字开头、特殊符号（如 -）

【命题趋势与实战策略】
- 考场秒杀：看到数字开头直接排除，看到关键字直接排除
- 高频陷阱：$ 合法但少见，_ 合法但不推荐

6. 以下关于 **JVM** 的说法，正确的是？

A. JVM 是 Java 编译器

B. 不同平台需要不同的 JVM

C. JVM 可以直接执行 Java 源代码

D. JVM 属于硬件设备

答案与深度解析 答案：**B.** 不同平台需要不同的 **JVM**

【核心认知框架】
- 题目本质：考查 Java 跨平台原理与 JVM 角色

fime专升本

- 关键区分点：JVM vs JDK vs JRE，字节码 vs 源代码
- 命题人意图：测试"Write Once, Run Any" 的实现机制

【选项原理深度拆解】

7. **A. JVM** 是 **Java** 编译器—错误（角色混淆）
   - 事实：编译器是 `javac`，JVM 是运行时环境
   - 输入输出：`javac` 输入.java 输出.class；JVM 输入.class 执行
   - 字节码证据：`javac` 生成常量池，JVM 加载常量池
8. **B.** 不同平台需要不同的 **JVM** —正确（跨平台核心）
   - 实现原理：Java 源代码编译成统一字节码，JVM 负责将字节码翻译成特定 OS 指令
   - 证据：Windows 需 Windows JVM，Linux 需 Linux JVM
   - **JNI**：JVM 通过 Java Native Interface 调用本地库
9. **C. JVM** 可以直接执行 **Java** 源代码—错误（需编译）
   - 流程：.java -> (javac) -> .class -> (JVM) -> 机器码
   - 例外：某些脚本引擎可执行源码，但标准 JVM 不行
   - 字节码：JVM 指令集操作码（如 `iload`）不对应源码语法
10. **D. JVM** 属于硬件设备—错误（软件层）
    - 定义：JVM 是运行在操作系统之上的软件虚拟机
    - 硬件交互：通过 OS 抽象层访问硬件，非直接硬件
    - 混淆点：Java 处理器（硬件实现 JVM）存在但极少见

【应背诵知识点】

- 跨平台原理：字节码统一 + 平台相关 JVM
- JVM 职责：加载、验证、执行、内存管理

【命题趋势与实战策略】

- 考场秒杀：JVM 是软件，负责运行，平台相关
- 高频陷阱：混淆编译器和虚拟机

11. 在 **Java** 中，下列哪个关键字用于导入包？

答案与深度解析 答案：**B. import**【核心认知框架】：考查包管理语法。【选项深度拆解】：

- **A. package**：错误。用于定义包，必须在文件首行。
- **B. import**：正确。编译期指令，告诉编译器类路径。
- **C. include**：错误。C/C++ 语法。
- **D. uses**：错误。Java 9 模块系统用 `requires`，无 uses 导入。

fifine专升本

- 字节码：import 不影响字节码，常量池存全限定名。

12. 下列哪个不是 **Java** 的基本数据类型？

答案与深度解析 答案：**C. String**【核心认知框架】：考查类型系统分类。【选项深度拆解】：

- **A/B/D**：错误。int/boolean/char 均为 8 大基本类型。
- **C. String**：正确。`java.lang.String` 是引用类型（类）。
- 内存模型：基本类型存栈值，String 存堆对象引用。
- 字节码：基本类型用 `iload`，引用类型用 `aload`。

13. 表达式 `10 \% 3` 的结果是？

答案与深度解析 答案：**C. 1**【核心认知框架】：考查算术运算符。【选项深度拆解】：

- 计算：10 除以 3 商 3 余 1。
- 字节码：`irem` 指令计算整数余数。
- 陷阱：负数取余结果符号与被除数相同。

14. 关于数组 **length** 属性的说法，正确的是？

答案与深度解析 答案：**B.** 数组的 **length** 表示数组可容纳的最大元素个数【核心认知框架】：考查数组属性。【选项深度拆解】：

- **A**：错误。length 是 final 字段，非方法（String 是 length() 方法）。
- **B**：正确。固定长度，创建时确定。
- **C**：错误。数组长度不可变。
- **D**：错误。二维数组 length 是第一维长度。
- 字节码：`arraylength` 指令获取长度。

15. 下列哪个修饰符可以使成员只能在本类中访问？

答案与深度解析 答案：**C. private**【核心认知框架】：考查访问控制权限。【选项深度拆解】：

- **A. public**：错误。全局可见。
- **B. protected**：错误。包内 + 子类可见。
- **C. private**：正确。仅本类可见。
- **D.** 默认：错误。包内可见。
- 字节码：访问标志 ACC_PRIVATE 限制访问。

16. 关于 **this** 关键字的描述，错误的是？

fifine专升本

答案与深度解析 答案：**C. this** 可以在静态方法中使用【核心认知框架】：考查 this 上下文。【选项深度拆解】：

- **C**：错误。静态方法属于类，无当前对象实例，无 this。
- **A/B/D**：正确。this 指代当前对象，可调用构造器/区分变量/返回。
- 字节码：静态方法无 `aload_0` (this) 参数。

17. 下列哪个类是所有 **Java** 异常类的父类？

答案与深度解析 答案：**C. Throwable**【核心认知框架】：考查异常体系结构。【选项深度拆解】：

- **C**：正确。Exception 和 Error 均继承自 Throwable。
- **A/B/D**：错误。均为 Throwable 子类。
- 字节码：`athrow` 指令要求操作数是 Throwable 子类。

18. 在 **try-catch-finally** 结构中，如果 **try** 块中有 **return**，那么 **finally** 块何时执行？

答案与深度解析 答案：**A.** 在 **return** 之前执行【核心认知框架】：考查控制流与 finally 语义。【选项深度拆解】：

- **A**：正确。finally 在 return 返回值前执行（可能修改返回值）。
- 字节码：`jsr` (Jump Subroutine) 或新实现保证 finally 先执行。
- 陷阱：finally 中 return 会覆盖 try 中 return。

19. 下列哪个集合类是有序且允许重复元素的？

答案与深度解析 答案：**C. ArrayList**【核心认知框架】：考查集合特性。【选项深度拆解】：

- **C**：正确。List 有序可重复。
- **A/B**：错误。Set 无序/有序但不可重复。
- **D**：错误。Map 是键值对。
- 底层：ArrayList 基于动态数组，索引有序。

20. 关于泛型方法，说法正确的是？

答案与深度解析 答案：**B.** 泛型方法的类型参数可以在返回值前声明【核心认知框架】：考查泛型语法。【选项深度拆解】

- **B**：正确。`<T> T method()`。
- **A**：错误。普通类也可定义泛型方法。
- **C**：错误。可以是静态。

ffine专升本

- **D**：错误。参数类型可以是 T。
- 字节码：Signature 属性记录泛型信息。

21. 在 **Java** 中，实现多线程可以通过继承哪个类？

答案与深度解析 答案：**B. Thread** 【核心认知框架】：考查线程创建方式。【选项深度拆解】

- **B**：正确。继承 Thread 重写 run。
- **A**：错误。Runnable 是接口。
- **C/D**：错误。无此类。
- 对比：实现 Runnable 更优（避免单继承限制）。

22. 关于 **synchronized** 关键字的说法，错误的是？

答案与深度解析 答案：**C.** 可以修饰类 【核心认知框架】：考查同步锁范围。【选项深度拆解】

- **C**：错误。不能修饰类，可修饰静态方法（锁 Class 对象）。
- **A/B**：正确。可修饰方法/代码块。
- **D**：正确。保证原子性/可见性（通过 Monitor）。
- 字节码：`monitorenter/monitorexit` 或 ACC_SYNCHRONIZED。

23. 下列哪个类用于将字节流转换为字符流？

答案与深度解析 答案：**A. InputStreamReader** 【核心认知框架】：考查 IO 流转换。【选项深度拆解】

- **A**：正确。字节 -> 字符（解码）。
- **B**：错误。缓冲字符流。
- **C**：错误。文件字符流。
- **D**：错误。字符 -> 字节（编码）。
- 原理：使用 Charset 进行编码转换。

24. 关于序列化的说法，正确的是？

答案与深度解析 答案：**B.** 实现 **Serializable** 接口的类才能序列化 【核心认知框架】：考查对象持久化。【选项深度拆解】

- **B**：正确。标记接口。
- **A**：错误。需实现接口。
- **C**：错误。static 属于类，不序列化。

fifine专升本

- **D**：错误。transient 修饰的不序列化。
- 字节码：`writeObject` 检查 serialVersionUID。

25. 在 **JDBC** 中，哪个方法用于执行 **INSERT**、**UPDATE**、**DELETE** 操作？

答案与深度解析 答案：**B. executeUpdate()**【核心认知框架】：考查 JDBC API。【选项深度拆解】

- **B**：正确。返回受影响行数。
- **A**：错误。执行查询，返回 ResultSet。
- **C**：错误。执行任意 SQL，返回 boolean。
- **D**：错误。批量执行。
- 原理：DML 操作不返回结果集。

26. 关于 **BorderLayout** 的说法，正确的是？

答案与深度解析 答案：**A.** 组件可以放在 **CENTER**、**NORTH**、**SOUTH**、**EAST**、**WEST** 五个位置 【核心认知框架】：考查布局管理器。【选项深度拆解】

- **A**：正确。5 个区域。
- **B**：错误。每个区域只能放一个组件。
- **C**：错误。JFrame 默认 BorderLayout。
- **D**：错误。默认 CENTER。
- 原理：边界布局，中间组件拉伸。

fifine专升本

27. 在事件处理中，**ActionEvent** 对应的监听器接口是？

答案与深度解析 答案：**A. ActionListener** 【核心认知框架】：考查事件监听模型。【选项深度拆解】

- **A**：正确。动作事件。
- **B/C/D**：错误。分别对应鼠标/键盘/窗口。
- 方法：`actionPerformed(ActionEvent e)`。
- 原理：观察者模式实现。

28. 在 **TCP** 编程中，用于监听客户端连接的类是？

答案与深度解析 答案：**B. ServerSocket**【核心认知框架】：考查网络编程 API。【选项深度拆解】

- **B**：正确。服务端监听端口。
- **A**：错误。客户端发起连接。

- **C/D**：错误。UDP 协议。
- 方法：`accept()` 阻塞等待连接。
- 原理：三次握手建立连接。

二、判断题（本大题共 10 小题，每小题 2 分，共 20 分）

1. **Java** 是纯面向对象的语言，所有代码都必须写在类中。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查语言范式。【深度解析】：
- 事实：Java 中方法、变量必须属于类或接口。
- 例外：Java 8+ 支持 Lambda，但本质仍是对象（函数式接口实例）。
- 对比：C++ 允许全局函数，Java 不允许。
- 字节码：所有方法字节码均在 class 文件结构中。

2. 静态方法可以直接调用非静态成员变量。（　）

答案与深度解析 答案：× **(B)** 【核心认知框架】：考查静态上下文。【深度解析】：
- 原理：静态方法无 this 引用，非静态变量依赖实例存在。
- 编译错误：`non-static variable cannot be referenced from a static context`。
- 字节码：静态方法无 `aload_0`，无法访问实例字段 `getfield`。
- 解决：创建对象后通过对象引用访问。

3. 子类构造方法必须显式调用父类构造方法。（　）

答案与深度解析 答案：× **(B)** 【核心认知框架】：考查构造器链。【深度解析】：
- 规则：若未显式调用，编译器隐式插入 `super()`。
- 例外：若父类无无参构造，必须显式调用 `super(args)`。
- 字节码：构造器首行必为 `invokespecial` 调用父类 `<init>`。
- 陷阱：”显式” 二字错误，可以是隐式。

4. 接口中的方法默认都是 **public abstract** 的。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查接口规范。【深度解析】：
- **JLS**：§9.4 规定接口方法隐含 public abstract。
- **Java 8+**：支持 default/static 方法，但抽象方法仍默认 public。
- 字节码：接口方法标志 ACC_PUBLIC | ACC_ABSTRACT。
- 例外：private 方法（Java 9+）用于默认方法内部复用。

fifine专升本

5. 抛出异常的语句后面不能再有其他语句。（　）

答案与深度解析 答案：× **(B)** 【核心认知框架】：考查控制流可达性。【深度解析】：
- 事实：`throw e;` 后代码不可达，但 `try-catch` 块后可以。
- 编译错误：同一块内 `throw` 后代码报 `unreachable statement`。
- 例外：若在条件分支中，非必然抛出的路径后可有代码。
- 字节码：`athrow` 指令后若无跳转，后续字节码无效。

6. **HashMap** 允许使用 **null** 作为键和值。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查集合规范。【深度解析】：
- 规范：HashMap 允许一个 null 键，多个 null 值。
- 对比：Hashtable 不允许 null。
- 原理：null 键 hash 值定义为 0，存放在数组第一个位置。
- 陷阱：TreeMap 不允许 null 键（需比较）。

7. 泛型在编译时会进行类型擦除，因此运行时无法获取泛型类型信息。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查泛型实现机制。【深度解析】
- 机制：类型擦除（Type Erasure），泛型参数替换为 Object 或边界。
- 例外：通过反射获取 Class 的 GenericSuperclass 可保留部分信息。
- 字节码：Signature 属性保留泛型签名，但运行时对象无泛型类型。
- 陷阱：`new T()` 非法，因运行时不知 T 是何类。

8. 线程的 **yield()** 方法会使线程进入阻塞状态。（　）

答案与深度解析 答案：× **(B)** 【核心认知框架】：考查线程状态。【深度解析】
- 事实：yield 使线程从运行态回到就绪态，非阻塞。
- 作用：提示调度器让出 CPU，同优先级线程可执行。
- 状态：RUNNABLE -> RUNNABLE (内部调度)。
- 对比：`wait()` 进入 WAITING 阻塞状态。

9. **File** 类既可以操作文件，也可以操作目录。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查 File 类功能。【深度解析】
- 功能：File 是抽象路径名，可代表文件或目录。
- 方法：`mkdir()`, `isDirectory()`, `listFiles()`。
- 原理：操作系统路径抽象，不区分文件/目录类型，仅元数据不同。
- 注意：不负责读写内容（需 IO 流）。

fifine专升本

10. **PreparedStatement** 可以防止 **SQL** 注入攻击。（　）

答案与深度解析 答案：√ **(A)** 【核心认知框架】：考查 JDBC 安全。【深度解析】
- 原理：预编译 SQL 结构，参数作为数据而非指令执行。
- 对比：Statement 拼接字符串易受注入。
- 机制：数据库端解析 SQL 模板，参数绑定占位符。
- 最佳实践：所有用户输入查询均应用 PreparedStatement。

三、填空题（本大题共 5 小题，每空 2 分，共 20 分）

1. **Java** 的注释有三种：单行注释使用 _____，多行注释使用 _____，文档注释使用 _____。

答案与深度解析 答案：//、/* */、/** */ 【核心认知框架】：考查源代码文档化。【深度解析】
- 单行：`//`，编译器忽略至行末。
- 多行：`/* */`，可跨行，不可嵌套。
- 文档：`/** */`，可被 `javadoc` 工具提取生成 API 文档。
- 字节码：注释不出现在.class 文件中（除 SourceFile 属性）。
- 陷阱：`/* // */` 是多行注释，`//` 无效。

2. 在 **Java** 中，使用 _____ 关键字定义抽象方法，使用 _____ 关键字定义接口。

答案与深度解析 答案：**abstract**、**interface** 【核心认知框架】：考查抽象机制。【深度解析】
- **abstract**：修饰方法无方法体，强制子类实现。
- **interface**：定义接口类型，隐含 abstract/public。
- 字节码：抽象方法标志 ACC_ABSTRACT，无字节码指令。
- 限制：abstract 类可含具体方法，接口 Java 8+ 可含 default 方法。

3. 异常处理中，捕获异常的关键字是 _____，抛出异常的关键字是 _____。

答案与深度解析 答案：**catch**、**throw** 【核心认知框架】：考查异常控制流。【深度解析】
- **catch**：声明异常处理块，匹配异常类型。
- **throw**：实例化异常对象并抛出。
- 区分：`throws` 用于方法签名声明可能抛出的异常。

fifine专升本

- 字节码：异常表（Exception Table）定义 catch 范围，`athrow` 指令抛出。

4. 集合框架中，实现 **List** 接口的常用类有 _____ 和 _____。

答案与深度解析 答案：**ArrayList**、**LinkedList (或 Vector)** 【核心认知框架】：考查集合体系。【深度解析】

- **ArrayList**：动态数组，随机访问快 O(1)，插入删除慢 O(n)。
- **LinkedList**：双向链表，插入删除快 O(1)（已知节点），访问慢 O(n)。
- **Vector**：线程安全旧类，不建议使用。
- 选择：读多用 ArrayList，写多用 LinkedList。

5. 线程启动后，执行的是 _____ 方法中的代码，使当前线程暂停指定时间的方法是 _____。

答案与深度解析 答案：**run()**、**sleep()** 【核心认知框架】：考查线程生命周期。【深度解析】

- **run**：线程任务主体，`start()` 后 JVM 调用 `run()`。
- **sleep**：静态方法，`Thread.sleep(ms)`，不释放锁。
- 区别：直接调用 `run()` 只是普通方法调用，非多线程。
- 字节码：`invokestatic` 调用 sleep，`invokevirtual` 调用 run。

四、程序运行结果题（本大题共 5 小题，每小题 6 分，共 30 分）

1. 写出下列程序的输出结果：

```java
public class TestA {
public static void main(String[] args) {
int i = 0;
while (i < 5) {
i++;
if (i == 3) continue;
System.out.print(i + " ");
}
}
}
```

fifine专升本

答案与深度解析 答案：**1 2 4 5** 【核心认知框架】：考查循环控制与 continue 语义。【执行流程分析】

- **i=0**：进入循环，i++ -> 1，!=3，输出 1。
- **i=1**：i++ -> 2，!=3，输出 2。
- **i=2**：i++ -> 3，==3，continue（跳过输出），回到 while 判断。
- **i=3**：i++ -> 4，!=3，输出 4。
- **i=4**：i++ -> 5，!=3，输出 5。
- **i=5**：while 判断 false，结束。
- 字节码：`goto` 指令实现 continue 跳转至循环条件判断处。

2. 写出下列程序的输出结果：

```java
class Base {
public Base() { System.out.print("Base "); }
public void show() { System.out.println("Base show"); }
}
class Derived extends Base {
public Derived() { System.out.print("Derived "); }
public void show() { System.out.println("Derived show"); }
}
public class TestB {
public static void main(String[] args) {
Base obj = new Derived();
obj.show();
}
}
```

fifine专升本

答案与深度解析 答案：**Base Derived Derived show** 【核心认知框架】：考查构造器链与多态。【执行流程分析】

- 初始化：new Derived() -> 先调 Base 构造 -> 输出"Base " -> 再调 Derived 构造 -> 输出"Derived "。
- 多态：`obj` 编译类型 Base，运行类型 Derived。
- 调用：`obj.show()` 动态绑定到 Derived.show() -> 输出"Derived show"。
- 字节码：构造器隐式 `super()`，方法调用 `invokevirtual` 动态分派。

3. 写出下列程序的输出结果：

```java
public class TestC {
public static void main(String[] args) {
```

```
Integer a = 100;
Integer b = 100;
Integer c = 200;
Integer d = 200;
System.out.println(a == b);
System.out.println(c == d);
}
}
```

答案与深度解析 答案：**true false**【核心认知框架】：考查整数缓存池。【执行流程分析】

- 装箱：`Integer.valueOf(int)`。
- 缓存：-128 到 127 复用对象。
- **a==b**：100 在缓存内，引用相同，true。
- **c==d**：200 超出缓存，new 对象，引用不同，false。
- 字节码：`invokestatic Integer.valueOf`，缓存逻辑在方法内。

4. 写出下列程序的输出结果：

```java
public class TestD {
public static void main(String[] args) {
try {
int[] arr = {10, 20, 30};
System.out.println(arr[1]);
System.out.println(arr[5]);
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Index Error");
} catch (Exception e) {
System.out.println("Other Error");
} finally {
System.out.println("Finally Done");
}
System.out.println("End");
}
}
```

答案与深度解析 答案：

```
20
```

fifine专升本

```
Index Error
Finally Done
End
```

【核心认知框架】：考查异常捕获顺序与 finally。【执行流程分析】

- **arr[1]**：输出 20。
- **arr[5]**：抛 ArrayIndexOutOfBoundsException。
- 捕获：第一个 catch 精确匹配，输出"Index Error"。
- **finally**：必执行，输出"Finally Done"。
- 后续：异常已处理，继续执行，输出"End"。
- 字节码：异常表定义捕获范围，finally 通过 jsr 或新机制实现。

5. 写出下列程序的输出结果：

```java
import java.util.*;
public class TestE {
public static void main(String[] args) {
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
Iterator<String> it = list.iterator();
while (it.hasNext()) {
String s = it.next();
if (s.equals("B")) it.remove();
}
System.out.println(list);
}
}
```

答案与深度解析 答案：**[A, C]** 【核心认知框架】：考查迭代器安全删除。【执行流程分析】

- 遍历：Iterator 遍历列表。
- 删除：`it.remove()` 安全删除当前元素，修改 modCount。
- 对比：若用 `list.remove()` 会抛 ConcurrentModificationException。
- 结果：B 被移除，剩 A, C。
- 字节码：Iterator 接口方法调用，内部维护 expectedModCount。

fifine专升本

五、编程题（本大题共 2 小题，每小题 20 分，共 40 分）

1. 设计一个员工工资系统

答案与深度解析 答案：见代码实现 【核心认知框架】：考查抽象类、多态、数组遍历。【设计思路深度解析】

- 抽象类 **Employee**：定义通用属性 name, id 和抽象方法 getSalary。
- 子类 **Manager/Worker**：扩展特有属性，实现工资计算逻辑。
- 多态：`Employee[]` 数组存储子类对象，统一调用 getSalary。
- 内存模型：数组存引用，对象存堆，多态调用动态绑定。
- 最佳实践：遵循开闭原则，易于扩展新员工类型。

【参考代码】

```java
abstract class Employee {
    private String name;
    private String id;
    public Employee(String name, String id) {
        this.name = name; this.id = id;
    }
    public abstract double getSalary();
    public String getName() { return name; }
}

class Manager extends Employee {
    private double bonus;
    public Manager(String name, String id, double bonus) {
        super(name, id); this.bonus = bonus;
    }
    public double getSalary() { return 8000 + bonus; }
}

class Worker extends Employee {
    private double hoursWorked, hourlyRate;
    public Worker(String name, String id, double h, double r) {
        super(name, id); this.hoursWorked = h; this.hourlyRate = r;
    }
    public double getSalary() { return hoursWorked * hourlyRate; }
}

```

fifine专升本

```java
public class Main {
    public static void main(String[] args) {
        Employee[] emps = new Employee[4];
        emps[0] = new Manager("M1", "001", 2000);
        emps[1] = new Manager("M2", "002", 3000);
        emps[2] = new Worker("W1", "003", 100, 50);
        emps[3] = new Worker("W2", "004", 120, 50);
        double total = 0;
        for (Employee e : emps) {
            double s = e.getSalary();
            total += s;
            System.out.println(e.getName() + ": " + s);
        }
        System.out.println("Total: " + total);
    }
}
```

【考点延伸】

- 多态：`e.getSalary()` 运行时绑定具体实现。
- 抽象：强制子类实现工资计算，规范接口。
- 扩展：新增 Intern 类只需继承 Employee，无需改 main。

2. 实现一个简单的银行账户类

答案与深度解析 答案：见代码实现 【核心认知框架】：考查封装、异常处理、业务逻辑。【设计思路深度解析】

- 封装：私有属性，公共方法控制访问。
- 异常：非法参数抛 IllegalArgumentException，余额不足抛自定义异常或运行时异常。
- 线程安全：简单实现非线程安全，生产环境需 synchronized。
- 健壮性：检查金额正负，防止逻辑错误。

【参考代码】

```java
class BankAccount {
    private String accountNumber;
    private String owner;
    private double balance;
    public BankAccount(String num, String owner, double balance) {
        this.accountNumber = num; this.owner = owner; this.balance =
```

fifine专升本

```java
            balance;
    }
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("□□□□□□□");
        balance += amount;
    }
    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("□□□□□□□");
        if (amount > balance) throw new IllegalArgumentException("□□□□
            ");
        balance -= amount;
    }
    public double getBalance() { return balance; }
    public String toString() {
        return owner + "(" + accountNumber + "): " + balance;
    }
}

public class TestBank {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("123", "Alice", 1000);
        try {
            acc.deposit(500);
            acc.withdraw(200);
            System.out.println(acc);
            acc.withdraw(2000); // □□□□
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

【考点延伸】

- 异常：使用异常处理业务规则 violation。
- 封装：balance 不可直接修改，保证数据一致性。
- 扩展：可添加交易记录列表，实现审计功能。

——试卷结束，祝考试顺利——

fifine专升本